



SECURITY AUDIT

Lista Dao

Collateral Yield

FINAL REPORT

June 2026

BAILSEC.IO



Disclaimer:

Security assessment projects are time-boxed and reliant on information provided by the client, its affiliates, or its partners. The findings in this report should not be considered a comprehensive list of security issues, flaws, or defects in the target system or codebase, and apply only to the specific commit hash, version, or deployment reviewed at the time of the audit. Any subsequent modification, redeployment, or upgrade is outside the scope of this report and may invalidate its findings.

The audit does not constitute investment, legal, financial, regulatory, or tax advice, nor an endorsement of any project or team. It is based on a limited review of materials provided and is delivered on an "as-is," "where-is," and "as-available" basis. Use of blockchain technology is subject to unknown risks and flaws. The scope of this assessment is technical and contract-level; unless explicitly stated otherwise, it does not cover economic or game-theoretic risks, MEV, oracle manipulation, governance attacks, centralization or admin-key risks, off-chain components, front-end code, or the behavior of third-party protocols, dependencies, libraries, or services interacted with by the audited code.

We make no warranties, express or implied, regarding the accuracy, reliability, completeness, or availability of the report or any associated services, and disclaim all implied warranties including merchantability, fitness for a particular purpose, and non-infringement. We assume no responsibility for any product, service, software, code, or material referenced by, linked to, or accessible through the report.

This report is prepared solely for the contract owner and may not be relied upon by any third party. The contract owner is responsible for making their own decisions based on the report and should seek additional professional advice as needed. To the maximum extent permitted by law, our total aggregate liability arising from or related to this audit shall not exceed the fees actually paid for the engagement. The contract owner agrees to indemnify and hold harmless the audit firm and its personnel from any claims, damages, expenses, or liabilities arising from use of or reliance on the report or the audited smart contract.

This disclaimer and any dispute arising from the audit shall be governed by the laws of Wyoming (USA), without regard to its conflict-of-laws principles. By engaging in this audit, the contract owner acknowledges and agrees to these terms.

1. Project Details

Important: Please ensure that the deployed contract matches the source-code of the last commit hash.

Project	Lista Dao – Collateral Yield – Audit Report
Website	Lista.org
Language	Solidity
Methods	Manual Analysis
Github repository	https://github.com/lista-dao/moolah/tree/7f78b13c23634bcb1a8450286e5ba40438a252a5/src/moolah-vault
Resolution 1	https://github.com/lista-dao/moolah/tree/8e00f37389d4894aaf6b5de19b91fd7ef64ca5f0

2. Detection Overview

Severity	Found	Resolved	Partially Resolved	Acknowledged (no change made)	Failed resolution	Open
High	1	1				
Medium	3			3		
Low	8	1	1	6		
Informational	10	3		7		
Governance	2			2		
Total	24	5	1	18		

2.1 Detection Definitions

Severity	Description
High	The problem poses a significant threat to the confidentiality of a considerable number of users' sensitive data. It also has the potential to cause severe damage to the client's reputation or result in substantial financial losses for both the client and the affected users.
Medium	While medium level vulnerabilities may not be easy to exploit, they can still have a major impact on the execution of a smart contract. For instance, they may allow public access to critical functions, which could lead to serious consequences.
Low	Poses a very low-level risk to the project or users. Nevertheless the issue should be fixed immediately
Informational	Effects are small and do not post an immediate danger to the project or users
Governance	Governance privileges which can directly result in a loss of funds or other potential undesired behavior

3. Detection

Global

Issue_01	Governance Privilege
Severity	Governance
Description	Currently, the contract exposes one or more governance privileges which can result in unexpected operations and potential loss of funds. Furthermore, the system also relies on the correctness of the underlying mechanics. For example, if the provider gets deregistered from a market, it will lock all collateral. Multiple things can become problematic if settings are adjusted within SlisBNBProvider, SlisBNBXMinter and Moolah.
Recommendations	Consider ensuring that the underlying configurations are properly handled.
Comments / Resolution	Acknowledged

CollateralYieldVault

The `CollateralYieldVault` contract is an ERC4626 vault contract that allows users to deposit SlisBNB which then gets forwarded to the `SlisBNBProvider` and deposited into a specified Moolah Market.

Due to the interaction with `SlisBNBProvider` and further `SlisBNBXMinter`, SlisBNBX tokens are being minted to the vault which allow for launchpad access. The `CollateralYieldVault` contract exposes a `setDelegateTarget` function that allows governance to determine an address which becomes the delegatee for SlisBNBX and mints the corresponding SlisBNBX amount to this address, which then allows this address for participating in the launchpad.

Appendix: SlisBNBProvider

`SlisBNBProvider` is the integration layer through which `CollateralYieldVault` holds its slisBNB as Moolah collateral: the vault supplies and withdraws exclusively via `supplyCollateral/withdrawCollateral` and reads `userTotalDeposit[vault]` directly as its `totalAssets[]`, so the vault's entire solvency surface reduces to this contract's accounting.

`_syncPosition` mirrors `MOOLAH.position[id, vault].collateral` into `userMarketDeposit[vault][id]` and `userTotalDeposit[vault]` using raw, exact deltas, so for the vault's single market the invariant `userTotalDeposit[vault] == position[id, vault].collateral` holds with no rounding drift, and the `slisBNBx` delegation performed by `_rebalanceUserLp` (through `SlisBNBxMinter`) is a launchpool cost-sink that is fully decoupled from this figure and never moves `totalAssets`.

Appendix: SlisBNBXMinter

`SlisBNBxMinter` is the launchpool-participation layer invoked by `SlisBNBProvider` on every collateral supply/withdraw (via `rebalance`): for the vault it derives `stakedAmount = getUserBalanceInBnb[vault]` from the provider's collateral, applies the module discount to obtain `amountAfterDiscount`, and splits it into a fee leg `newReservedLp = amountAfterDiscount.mulDivUp[feeRate, DENOMINATOR]` minted to the shared `mpcWallets` and a user leg `newUserLp` delegated to the vault's `delegateTarget`, while tracking `userTotalBalance[vault]`, `delegation[vault]`, and `userModuleBalance[vault][provider]`.



Privileged Functions

- setFee
- setFeeRecipient
- setWhitelist
- setDelegateTarget
- sweep
- emergencyWithdraw
- pause
- unpause
- increaseVaultAssets

Invariants

INV 1: Only whitelisted callers can deposit/mint for whitelisted receivers

INV 2: Each totalAssets increase must mint a fee as shares

Issue_02	Attacker can drain vault and brick withdrawals due to incorrect totalAssets tracking
Severity	High
Description	The <code>CollateralYieldVault</code> tracks the total assets of the vault based on the <code>userTotalDeposit</code> mapping from the <code>SlisBNBProvider</code>. <pre>function totalAssets() public view override returns (uint256) { return PROVIDER.userTotalDeposit(address(this)); }</pre> This is inaccurate since the <code>userTotalDeposit</code> mapping represents the total slisBNB tokens supplied as collateral across all Moolah markets that have slisBNB as its collateral. This can be observed in the <code>SlisBNBProvider._syncPosition</code> function which tracks user deposits in two different mappings: <code>userTotalDeposit</code> and <code>userMarketDeposit</code>. Mapping <code>userMarketDeposit</code> tracks the collateral supplied to a specific market. <pre>if (userMarketSupplyCollateral >= userMarketDeposit[account][id]) { uint256 depositAmount = userMarketSupplyCollateral - userMarketDeposit[account][id]; userTotalDeposit[account] += depositAmount; } else { uint256 withdrawAmount = userMarketDeposit[account][id] - userMarketSupplyCollateral; userTotalDeposit[account] -= withdrawAmount; } userMarketDeposit[account][id] = userMarketSupplyCollateral;</pre> In function <code>totalAssets</code>, the <code>CollateralYieldVault</code> should only track the collateral supplied to the slisBNB/BNB Moolah market and not all markets. Since it overestimates the supplied collateral, this can allow a malicious actor to inflate the share price and drain the vault through a crafted attack. Example:

- Assume two users: Malicious Alice and honest Bob
- Bob deposits 100e18 slisBNB to the CollateralYieldVault and receives 100e18 shares.
- Global State: totalAssets = 100e18, totalSupply = 100e18
- Alice observes this and deposits 100e18 slisBNB to receive 100e18 shares.
- Global State: totalAssets = 200e18, totalSupply = 200e18
- Alice supplies 200e18 slisBNB directly to the provider for a different market with collateral as slisBNB.
- Global State: totalAssets = 400e18, totalSupply = 200e18
- As per the vault's totalAssets, this means both Alice and Bob should each receive 200e18 assets on redemption.
- Alice redeems the 100e18 shares to receive the 200e18 assets from the slisBNB/BNB market.
- When Bob attempts to redeem, the transaction fails since the provider would attempt to withdraw more assets than what were supplied to the slisBNB/BNB market.

Direct supply of slisBNB collateral to a different moolah market can also brick the sweep function since some users will never be able to redeem certain portion or all of their shares, leaving a non-zero totalSupply.

Impact 1:

Late depositors become insolvent, their redemptions compute an assets amount that exceeds the remaining market-id collateral and `MOOLAH.withdrawCollateral` underflow-reverts, so they cannot withdraw at all; their principal has been paid out to the earlier redeemer.

Impact 2:

Zero-net-cost griefing/extraction: the attacker recovers their entire outlay [deposit + donation] by redeeming inflated-value shares against the real collateral [which now includes victims' deposits], netting ≈ 0 while the victims lose, so the attack is economically free whenever a

	victim deposits after the inflation (and can be made deterministic by front-running a pending deposit). Impact 3: Vault NAV / accounting corruption: <code>totalAssets()</code> permanently overstates the withdrawable backing by the stranded D, mispricing every subsequent deposit/redeem and (with fee > 0) charging performance fee on phantom "interest" (fee accrual amplified).
Recommendations	Consider using the storage variable which is mapped to the exact market.
Comments / Resolution	Resolved, instead of the recommended storage variable which is mapped to the exact market (<code>userMarketDeposit</code>), the contract now uses the direct underlying MOOLAH position: <code>MOOLAH.position(marketParams.id(), address(this)).collateral;</code> While this reflects the correct balance, it can become not 100% accurate as per what's withdrawable from the <code>SlisBNBProvider</code>. The provider-recognized value is updated through <code>_syncPosition()</code> and includes this provider-registration branch: <pre>if (MOOLAH.providers[id, TOKEN] != address(this)) { userMarketSupplyCollateral = 0; }</pre> The new vault <code>totalAssets()</code> does not apply that branch. Therefore, if the Moolah market still records collateral for the vault but <code>SlisBNBProvider</code> is no longer the registered provider, the vault can still report <code>totalAssets > 0</code>. Moreso, Vault NAV can diverge from SlisBNBx reward/voting accounting

Issue_03	Orphaned rewards in case of zero supply will cause side-effects
Severity	Medium
Description	Whenever there is no supply, it can still happen that rewards are distributed. These would then be assigned to the virtual shares [1]. There are multiple side-effects from that behavior for example an attacker can frontrun the reward distribution and can cause an inflation attack to the vault which would brick it permanently or the reward amount can be simply fully allocated to the virtual share which would then brick future deposits until the sweep function is called (temporary impact only).
Recommendations	Consider ensuring that there is always a non-zero supply via a governance deposit.
Comments / Resolution	Acknowledged, the protocol added the following comment: <pre> /// @dev NOTICE: rewards injected while totalSupply == 0 accrue to the virtual shares. The deploy-time dead-seed /// first deposit keeps supply > 0, so this state is not reached in practice. </pre> It is true that <code>totalSupply()</code> will not reach 0 because of the dead-seed deposit. However, the comment only addresses the <code>totalSupply == 0</code> case; it does not resolve the underlying problem. If all real shareholders have exited the vault and rewards are then harvested, those rewards accrue to the dead-seed shares, which are unredeemable. The rewards therefore remain permanently stranded, so the issue still exists.

Issue_04	Max functionality does not reflect <code>msg.sender</code> whitelist
Severity	Low
Description	The <code>maxDeposit</code> and <code>maxMint</code> functions reflect the whitelist mechanism for the receiver but fail to reflect the whitelist mechanism for the <code>msg.sender</code>, essentially indicating that some deposits might be allowed while they would revert. Furthermore, this is true for <code>previewDeposit</code> and <code>previewMint</code>. Also these functions may return true while one underlying mechanism does not work (pause market, supply cap etc).
Recommendations	Consider mimicking the proper limitations.
Comments / Resolution	Partially fixed by ensuring the <code>msg.sender</code> is in the whitelist. The preview functionality does not implement any limitations. Also, the existing max functionality does not implement other limitations such as market pause.

Issue_05	Disabled whitelist on initialization enables whitelist bypass
Severity	Low
Description	The CollateralYieldVault implements whitelist checks across all the ERC4626 functions. However, since whitelist is disabled by default on initialization, any non-whitelisted actor can backrun initialization to deposit and receive shares. Furthermore, enabling the whitelist will freeze existing holders.
Recommendations	Consider adding the admin to the whitelist on initialization to activate the whitelist.
Comments / Resolution	Acknowledged - documented

Issue_06	Donations can break the vault indefinitely
Severity	Low
Description	Anyone can call <code>slisBNBProvider.supplyCollateral</code> on behalf of the vault and then call the <code>accrueFee</code> function. If that happens before the first deposit or if generally <code>supply = 0</code> at some point in time, the attacker essentially donated funds to the virtual reserves but at the same time it can also result in future depositors to receive no shares for their deposit due to the inflated divisor. A user can then afterwards execute a deposit such that <code>supply > 0</code> (the deposit will round received shares down) and as such it will disable the call to sweep but keep the poisoned exchange rate.
Recommendations	Consider executing the first deposit as governance during the deployment to make sure any donation will be captured.
Comments / Resolution	Acknowledged, the client will make a dead seed deposit on deployment. The <code>deposit()/mint()</code> have <code>shares == 0</code> and <code>assets == 0</code> guard.

Issue_07	Module setting update can result in vault operation reverts due to insufficient mint cap for MPC wallets
Severity	Low
Description	<code>CollateralYieldVault</code> holds user principal in <code>slisBNB</code> units. Deposits, BNB deposits, reward compounding, withdrawals, redemptions, and sweeps move <code>slisBNB</code> collateral through <code>SlisBNBProvider</code>. In the active production route, <code>SlisBNBProvider</code> does not mint or burn the reward/voting token itself. Instead, every provider position sync calls <code>SlisBNBxMinter.rebalance(account)</code>. For the vault, the account being rebalanced is <code>address[CollateralYieldVault]</code>. The minter reads the vault's provider-backed <code>slisBNB</code> position through <code>SlisBNBProvider.getUserBalanceInBnb(address[vault])</code>, converts that provider balance into BNB value through <code>StakeManager.convertSnBnbToBnb()</code>, then applies the module's <code>discount</code> and <code>feeRate</code>. The relevant minter state is: <pre> moduleConfig[module].discount moduleConfig[module].feeRate userModuleBalance[account][module].userPart userModuleBalance[account][module].feePart userTotalBalance[account] mpcWallets[i].balance mpcWallets[i].cap </pre> The vault's <code>slisBNB</code> principal amount does not depend on <code>SlisBNBx</code>. However, vault principal operations are still execution-coupled to <code>SlisBNBx</code> because provider supply and withdraw both call <code>_syncPosition()</code>, and <code>_syncPosition()</code> calls <code>SlisBNBxMinter.rebalance()</code>. <code>SlisBNBxMinter.updateModules()</code> lets <code>MANAGER</code> update <code>discount</code> and <code>feeRate</code> for an existing module. The function only writes the new config

into `moduleConfig`. It does not rebalance existing accounts.

Existing account state remains recorded under the old config:

```
userModuleBalance[account][module].userPart
userModuleBalance[account][module].feePart
```

A later rebalance recomputes the account's entire module balance under the new config:

```
stakedAmount = module.getUserBalanceInBnb(account)
amountAfterDiscount = stakedAmount * (DENOMINATOR - discount) /
DENOMINATOR
newReservedLp = amountAfterDiscount.mulDivUp(feeRate,
DENOMINATOR)
newUserLp = amountAfterDiscount - newReservedLp
```

This means the next user operation touching the vault provider position may execute a delayed config migration even if the vault's slisBNB principal movement is small or even directionally reducing. For example, a withdrawal can still require **additional reserved SlisBNBx minting** if `feeRate` increased or `discount` decreased since the last sync.

If the computed reserved part increases, the minter calls `_mintToMPCs()`. `_mintToMPCs()` requires enough MPC cap headroom. If cap is insufficient, the rebalance reverts. Because the rebalance is executed inside `SlisBNBProvider._syncPosition()`, the entire vault deposit, withdrawal, redemption, reward-compound, or sweep reverts.

The root cause is that a delayed, global sidecar-accounting transition is coupled to user-facing principal flows. The vault has no separation between "move slisBNB collateral" and "apply pending SlisBNBx config rebalance."

Impact 1:

A valid vault withdrawal or redemption can revert even when the vault

has enough slisBNB collateral in Moolah. The failure comes from SlisBNBx sidecar accounting, not from insufficient principal.

Impact 2:

A later vault operation can unexpectedly mint, burn, or redistribute SlisBNBx between the vault delegatee and MPC wallets due only to a prior config change, even if the underlying slisBNB position did not increase.

Impact 3:

Module disabling is lazy. If **discount** is set to **DENOMINATOR** and **feeRate** to zero, old SlisBNBx balances can remain until a successful rebalance. Reward or voting snapshots before that rebalance can still observe stale SlisBNBx balances.

Impact 4:

The first user or caller who touches the provider after a config update inherits the cost and liveness risk of applying the delayed transition.

Initial state:

1. Vault provider balance in BNB terms: 1,000 BNB
 - discount = 0%
 - feeRate = 10%
 - oldReservedLp = 100
 - oldUserLp = 900
2. **MANAGER** updates:
 - feeRate = 20%

	No account is rebalanced during the update, so the vault still has: - <code>userModuleBalance[vault][provider].feePart = 100</code> - <code>userModuleBalance[vault][provider].userPart = 900</code> 3. Later, a user withdraws a small amount. After the withdrawal, assume the remaining provider balance is still: - 999 BNB - Under the new config: - <code>newReservedLp = 999 * 20% = 199.8</code> rounded up - <code>newUserLp ≈ 799.2</code> - The withdrawal path now needs to mint roughly: - ~100 additional reserved SlisBNBx - to MPC wallets, while also burning user-part SlisBNBx from the delegatee. If MPC caps cannot absorb the additional reserved amount, <code>_mintToMPCs()</code> reverts, and the withdrawal reverts despite the vault reducing its slisBNB principal position.
Recommendations	Consider ensuring the cap is always sufficient. We recommend this in general to make sure withdrawals never get locked.
Comments / Resolution	Acknowledged - documented

Issue_08	Fee increase before harvesting will apply new fee on already entitled amount
Severity	Low
Description	The reward distribution is asynchronous: rewards can already be present but just not yet distributed. This opens a window where governance can increase the fee while there are already present rewards but not yet distributed. After the fee increase, the BOT then calls <code>increaseVaultAssets</code> which captures a larger fee than before.
Recommendations	Consider acknowledging this issue.
Comments / Resolution	Acknowledged - documented

Issue_09	Lack of zero share guard during deposit
Severity	Low
Description	The <code>depositBNB</code> function has the following check: <pre><i>if (shares == 0) revert ZeroAmount();</i></pre> This check is not present within the deposit function and therefore it could be possible that users receive zero shares from their deposits.
Recommendations	Consider implementing the same check within the deposit function.
Comments / Resolution	Fixed

Issue_10	Same-target guard can block delegation re-initialization after provider minter replacement
Severity	Low
Description	This issue has been identified due to a post-audit change which was not provided by Bailsec: <pre><i>if (target == delegateTarget) revert ErrorsLib.AlreadySet{}; // minter no-ops on same target; avoid misleading event</i></pre> The final version assumes that <code>target == delegateTarget</code> means the active minter is already configured. That assumption is only valid if <code>PROVIDER.slisBNBxMinter()</code> is immutable. In this architecture, the provider minter is read dynamically and can be re-set by provider governance. The vault-local <code>delegateTarget</code> and the minter-local <code>delegation[address(vault)]</code> are different storage domains. Equality against the vault-local variable does not prove that the currently active minter has matching delegation state. • <code>SlisBNBProvider.slisBNBxMinter()</code> is changed after the vault has already set a delegate target. • The intended delegate target remains the same address. • The new minter has no delegation state for <code>address(vault)</code>. • A manager attempts to re-apply the same delegate target through the vault. • Later provider sync or vault deposit/withdraw/compound triggers <code>SlisBNBxMinter.rebalance(address(vault))</code>. The vault's SlisBNBx can be minted to <code>address(vault)</code> or remain misrouted instead of being delegated to the intended launchpool address. If reward snapshots or Launchpool allocation rely on raw SlisBNBx holder balances, the vault may lose, delay, or misroute reward power.

	This does not directly change vault slisBNB principal or ERC4626 share accounting. The impact is reward/voting sidecar misrouting and operational recovery friction after minter replacement.
Recommendations	Consider removing this check.
Comments / Resolution	Acknowledged.

Issue_11	Redundant fee casting to uint96
Severity	Informational
Description	Within the <code>setFee</code> function, <code>newFee</code> is casted as follows: <pre>> uint96(newFee);</pre> This can be simplified if the parameter is directly uint96.
Recommendations	Consider making the parameter uint96.
Comments / Resolution	Fixed

Issue_12	Meta-transactions cannot work with vault deposit functionality
Severity	Informational
Description	The <code>deposit</code>, <code>mint</code> and <code>depositBNB</code> functions implement a whitelist check that ensures both the <code>msg.sender</code> and share receiver are whitelisted. <pre>function _checkWhiteList(address receiver) private view { require(isWhiteList[_msgSender()] && isWhiteList(receiver), ErrorsLib.NotWhiteList()); }</pre> However, requiring <code>msg.sender</code> to be whitelisted would mean that relayers performing gasless transactions for users would also need to be whitelisted, which would enable any user to bypass the whitelist through it. This creates a scenario where relayers cannot be provided whitelist access, disallowing a certain set of users relying on gasless transactions from participating in vault operations.
Recommendations	Consider acknowledging this finding if strict <code>msg.sender</code> validation is intended as per design
Comments / Resolution	Acknowledged - documented

Issue_13	Fee rounds down
Severity	Informational
Description	The fee is calculated as follows: <pre>> uint256 feeAssets = totalInterest.mulDiv(fee, WAD);</pre> This calculation truncates the remainder and as such, will round down the fee (potentially to zero in case of totalInterest dust). This is however not too impactful because the totalInterest usually is no dust because it can only ever be triggered by BOT rewarding (or donations).
Recommendations	Consider acknowledging this note.
Comments / Resolution	Acknowledged

Issue_14	Malicious actor can frontrun consume MPC wallet cap to grief collateral supply operations
Severity	Informational
Description	The SlisBNBProvider (utilizing SlisBNBxMinter) implements mint caps for MPC wallets. A malicious actor can frontrun consume these caps directly through the provider to disallow collateral supply occurring through the CollateralYieldVault. This blocks the deposit functionality. <pre>require(leftToMint == 0, ErrorsLib.EXCEED_MPC_CAP);</pre>
Recommendations	Consider acknowledging this finding and monitoring such a scenario to adjust the wallet caps accordingly.
Comments / Resolution	Acknowledged

Issue_15	Event Swept and AccrueInterest emitted even when no sweep and accrual occurs
Severity	Informational
Description	The event Swept is emitted regardless of whether the if condition is triggered for a sweep or not. <pre> if (amount > 0) { PROVIDER.withdrawCollateral(marketParams, amount, address(this), to); _updateLastTotalAssets(0); } emit Swept(to, amount); </pre> Similarly AccrueInterest is emitted even when no interest is accrued. <pre> if (feeShares != 0) _mint(feeRecipient, feeShares); emit AccrueInterest(newTotalAssets, feeShares); </pre>
Recommendations	Consider emitting the events when the sweep and fee accrual actually occur.
Comments / Resolution	Fixed

Issue_16	Inconsistent use of custom errors and string error messages
Severity	Informational
Description	The vault and harvester contracts inconsistently use custom errors and string error messages. For example in the vault, the emergencyWithdraw function uses a string error message while sweep function uses a custom error.
Recommendations	Consider acknowledging this note.
Comments / Resolution	Acknowledged

Issue_17	Dead seed deposit concerns
Severity	Informational
Description	The deploy script <code>script/collateralYieldVault/deploy_collateral_yield_vault.sol</code> makes a dead seed deposit (0.02 BNB) immediately after deployment to mitigate the inflation attack. Because the seed shares are minted to <code>0x...dEaD</code>, they can never be redeemed, so <code>totalSupply()</code> is permanently greater than zero. This raises two concerns. 1. <code>sweep()</code> can never be called, <code>sweep()</code> requires <code>totalSupply() == 0</code>: <pre>if (totalSupply() != 0) revert SharesOutstanding();</pre> Since the dead seed shares are never redeemable, <code>totalSupply()</code> can never return to zero, so <code>sweep()</code> is unreachable in production. 2. The inflation attack is still reachable via a pre-deployment donation: The proxy is deployed with new <code>ERC1967Proxy(...)</code>, i.e. <code>CREATE</code>, so its address is deterministic from the deployer address and nonce. An attacker can compute the vault address before it is deployed and pre-load a donation to it via <code>PROVIDER.supplyCollateral(onBehalf = predictedVault)</code>, this works because Moolah positions are keyed by address and require no code at that address. When the seed runs, the vault already sees the donation in <code>totalAssets()</code>. The script's only guard is: <pre>require(vault.totalSupply() == seedShares && seedShares > 0, "seed failed");</pre> This does not detect a degraded seed. If the attacker sizes the donation so the seed mints exactly 1 wei of shares, the check becomes <code>require(1 == 1 && 1 > 0)</code>, which passes, and the deploy completes in the degraded

	state. To degrade the seed to 1 wei, the donation must be just below the seed's slisBNB amount (A - 1): - A [seed slisBNB] = <code>convertBnbToSnBnb</code>(0.02 BNB) $\approx 0.0194e18$ - <code>donation</code> X = A - 1 $\approx 0.0194e18$ (~\$12) - <code>seedShares</code> = floor[A $\times$ (0+1) / (X+1)] = floor[A / A] = 1 wei The vault therefore starts in: - <code>totalSupply</code> = 1 - <code>totalAssets</code> = X + A $\approx 0.0388e18$ - <code>ER</code> = (totalAssets + 1) / (totalSupply + 1) $\approx 0.0194e18$ Any deposit below $\sim 0.0194e18$ slisBNB will revert due to rounding down to zero shares. The value can be raised by donating more funds into the vault through the PROVIDER. This is marked as Info because it is documented that in deployments with a dead seed deposit <code>sweep()</code> can never be called.
Recommendations	Consider acknowledging this issue as this code may be used with another deployment process.
Comments / Resolution	Acknowledged.

RewardHarvester

The **RewardHarvester** contract is a simple helper contract that claims rewards from the **ClisBNBLaunchPoolDistributor** contract and then distributes these rewards to the CollateralYieldVault via the **increaseVaultAssets** function.

The only functions are **harvest** and **rescue** and both can only be called by the bot.

Privileged Functions

- harvest
- rescue

Issue_18	MANAGER can drain all rewards from the harvester using rescue functionality
Severity	Governance
Description	The rescue function allows the MANAGER to withdraw BNB and any ERC20 tokens from the contract. Since ClisBNBLaunchPoolDistributor claims are permissionless, the MANAGER can claim rewards for all active epochs to the harvester contract and drain them using the rescue function.
Recommendations	Consider implementing a strong multi-sig setup to avoid such a scenario in case of key-compromise.
Comments / Resolution	Acknowledged-documented

Issue_19	Malicious users can frontrun harvest to profit from immediate share price increase
Severity	Medium
Description	The harvest function in RewardHarvester claims BNB rewards from the distributor and supplies them to the CollateralYieldVault using increaseVaultAssets. <pre><i>VAULT.increaseVaultAssets{ value: bnbBal }();</i></pre> Function increaseVaultAssets deposits BNB to mint slisBNB and immediately supplies it as collateral to the provider and ultimately the Moolah market. Due to this, the share price jumps immediately. Any malicious actor can sandwich harvest with deposit-withdraw transactions to profit from this increase. Since share price increases are predictable, this poses a threat to long-term participation in the vault. <pre><i>function increaseVaultAssets() external payable override onlyRole(BOT) nonReentrant whenNotPaused { if (msg.value == 0) revert ZeroAmount(); uint256 balBefore = IERC20(SLIS_BNB).balanceOf(address(this)); STAKE_MANAGER.deposit{ value: msg.value }(); uint256 slisDelta = IERC20(SLIS_BNB).balanceOf(address(this)) - balBefore; PROVIDER.supplyCollateral(marketParams, slisDelta, address(this), "");</i></pre>
Recommendations	Consider linearly unlocking claimed rewards over a specific duration in the vault.
Comments / Resolution	Acknowledged

Issue_20	Permissionless distributor claim could lead to unexpected vault asset increase on harvest
Severity	Medium
Description	The claim function in the ClisBNBLaunchPoolDistributor is permissionless. This allows anyone to perform claims on behalf of the RewardHarvester and send the BNB rewards to it. This allows a malicious actor to cause an unintended share price increase. Assume the following scenario: - Epoch IDs 1 to 5 are active in the distributor - BOT intends to claim rewards for epoch 1 and 2 through the harvest function and supply it to the vault. - Malicious actor frontruns the BOT's harvest with claims for epochs 3, 4 and 5. - Since the harvest function supplies the entire BNB balance of the contract to the vault, rewards from epochs 3, 4, 5 are forwarded as well. - This can lead to a sudden increase in share price compared to what the BOT/team expected.
Recommendations	Consider documenting this behavior.
Comments / Resolution	Acknowledged-documented

Issue_21	Paused moolah market can temporarily block harvest leading to inactive epochs and unclaimed rewards
Severity	Low
Description	Here is the execution path of the harvest function: RewardHarvester.harvest => CollateralYieldVault.increaseVaultAssets => SlisBNBProvider.supplyCollateral => MOOLAH.supplyCollateral If the Moolah market is paused, supplyCollateral reverts. This causes a revert for the entire execution path, disallowing the harvest from occurring. If the paused state is prolonged, it could lead to epochs becoming inactive and leading to unclaimed rewards in the ClisBNBLaunchPoolDistributor.
Recommendations	Consider acknowledging and keeping this in mind. In such a scenario, claims should be performed directly on the distributor to bring rewards to the harvester contract and supplied later on once the market is unpaused.
Comments / Resolution	Acknowledged-documented

Issue_22	Donation to rewarder can unexpectedly increase harvested amount
Severity	Informational
Description	Currently, the harvest function simply forwards the existing BNB balance of the contract: <pre><i>uint256 bnbBal = address(this).balance; if (bnbBal == 0) revert NothingToCompound(); if (bnbBal < minBNBOut) revert InsufficientReward(); VAULT.increaseVaultAssets{ value: bnbBal }();</i></pre> Anyone can simply donate BNB to the contract which would then unexpectedly increase the rewarded amount.
Recommendations	Consider acknowledging this note.
Comments / Resolution	Acknowledged

Issue_23	Incorrect <code>claimCount</code> value emitted in <code>Harvested</code>
Severity	Informational
Description	Event <code>Harvested</code> is emitted in the <code>harvest</code> function. However, it uses the <code>claims</code> array length as the <code>claimCount</code>. This is incorrect since the <code>claims</code> array can include already claimed epochs, which are skipped in the <code>harvest</code> function's for loop. <pre><i>event Harvested(uint256 claimCount, uint256 bnbInjected); emit Harvested(claims.length, bnbBal);</i></pre>
Recommendations	Consider tracking the actual claims using a counter and use it during emission. Alternatively, acknowledge this issue.
Comments / Resolution	Fixed

Issue_24	Non-native ERC20 tokens cannot be claimed without idle BNB
Severity	Informational
Description	The harvest function attempts to claim all provided epochs and this can include tokens which are simple ERC20 tokens, see implementation of the Distributor contract. If there are no idle BNB tokens, the harvest function would always revert. Therefore there are two workarounds to claim ERC20 tokens: a) Claim them via harvest while the contract has idle BNB and then recover b) Claim them via direct interaction with the distributor such that they are distributed to the Rewarder and then recover them.
Recommendations	Consider keeping that note in mind.
Comments / Resolution	Acknowledged